SPRINGFORGE: INTELLIGENT SPRING BOOT DEVELOPMENT ECOSYSTEM

25-26J-451

Project Proposal Report

Ariyasena M.K.M.M.

B.Sc. (Hons) Degree in Information Technology Specializing Software Engineering

Department of Information Technology

Faculty of Computing

Sri Lanka Institute of Information Technology,
Sri Lanka

August 2025

SPRINGFORGE: INTELLIGENT SPRING BOOT DEVELOPMENT ECOSYSTEM

25-26J-451

Project Proposal Report

B.Sc. (Hons) Degree in Information Technology Specializing Software Engineering

Department of Information Technology

Faculty of Computing

Sri Lanka Institute of Information Technology Sri Lanka

August 2025

Declaration

Declaration

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Ariyasena MKMM	I22562524	

Signature of the Supervisor:
Date: 28/08/2025

Signature of the Co-Supervisor: S.S. Kumarasinghe 27/08/25
Date:

Abstract

The AI-powered Spring Boot debugging assistant, an IntelliJ plugin, enhances the debugging process for Spring Boot applications by tackling complex runtime errors like dependency injection failures and bean misconfigurations, which traditional debuggers struggle to resolve efficiently. Utilizing large language models (LLMs) and retrieval-augmented generation (RAG), it analyzes stack traces and logs in real-time, pulling relevant insights from curated sources such as Spring Boot documentation and Stack Overflow. The plugin delivers precise, context-aware fix suggestions directly within the IntelliJ IDE, improving upon low-level tools like JetBrains AI, aiming to reduce debugging time by 30–50%, improve fix accuracy, and elevate developer productivity with an intuitive, seamless interface.

TABLE OF CONTENTS

Declaration i
Abstract ii
Table of Contents iii

List of figures iv

1. INTRODUCTION 1

1.1. Background and Literature Survey 1
1.2. Research Gap 3
1.3. Research Problem 4
2. Objectives 5

2.1. Main Objective 5
2.2. Specific Objectives 5
3. Methodology 6

3.1. Materials and Resources 6
3.1.1. Software: 6
3.1.2. Hardware: 6
3.1.3. Data: 7
3.2. Data Collection 7
3.2.1. RAG Knowledge Base: 7
3.2.2. Test Dataset: 7
3.3. Tasks and Subtasks 8
3.4. Anticipated Outcomes 9
3.5. Architecture Diagram 10
4. Project Requirements 11

4.1. Functional Requirements 11
4.2. Non-Functional Requirements 11
4.3. User Requirements 11
4.4. System Requirements 11
4.5. Use Cases (Tentative) 11
4.6. Test Cases (Tentative) 12
4.7. Wireframes (Tentative) 12
4.8. Work Breakdown Structure 13
4.9. Gantt Chart 13
References 14

LIST OF FIGURES

Figure 1.1. Comparison of Debugging Tools and Features 1
Figure 3.1. Architecture Diagram 10
Figure 4.1. Work Breakdown Structure 13
Figure 4.2. Gantt Chart 13

1. INTRODUCTION

1.1. Background and Literature Survey

Debugging Spring Boot applications presents unique challenges due to the framework's complex lifecycle processes, including dependency injection and bean initialization. A systematic review of 43 debugging intervention studies reveals that ineffective debugging tools reduce developer productivity and contribute to increased dropout rates in computer science education [1]. As illustrated in Figure 1, existing debugging solutions vary significantly in their capabilities and approaches to handling Spring Boot-specific runtime errors.

Recent advancements in automated program repair (APR) leverage deep learning to address debugging challenges, treating APR as a neural machine translation task to generate fix suggestions from bug-fix datasets [2]. A comprehensive survey of 63 LLM-based APR systems from 2022-2025 classifies them into four design paradigms: prompting, procedural pipelines, and agentic frameworks, highlighting trade-offs such as latency in agentic approaches [4]. Tools like FixMiner mine actionable fix patterns using Rich Edit Scripts, capturing AST-level context to cluster similar patches, correctly fixing 26 bugs in the Defects4J benchmark with 81% of plausible patches being correct [5].

Features	Generic AI Chatbots	GitHub Copilot	JetBrains AI	Proposed Feature
Runtime Error Capture	✗	✗	✓	✓
RAG Support	✗	✓	✗	✓
Accuracy for Frameworks	⚠ Generic	⚠ Moderate	⚠ Generic	✓ Springboot aware
Fix Suggestions	⚠ Limited	⚠ Limited	⚠ Basic	✓ Automated, verified

Figure 1.1. Comparison of Debugging Tools and Features

As shown in Figure 1, current debugging tools can be categorized into several approaches. Generic AI Chatbots (e.g., ChatGPT or similar) provide broad assistance but lack runtime error capture and RAG support, resulting in generic accuracy for frameworks and limited fix suggestions. GitHub Copilot offers multi-IDE integration with RAG support for code suggestions but does not handle runtime error capture, providing only moderate framework accuracy and limited fixes without Spring Boot-specific context. JetBrains AI represents a more targeted IDE tool, capturing runtime errors but operating at a low level with generic accuracy for frameworks like Spring Boot and only basic fix suggestions, lacking RAG for enhanced, grounded insights.

Retrieval-Augmented Generation (RAG) systems address LLM limitations by integrating external knowledge bases, reducing hallucination, and providing up-to-date, domain-specific information [3]. Enhanced RAG systems with quantified influence measures achieve average response accuracy of 93.4% in domain-specific applications [10]. Agentic frameworks like VulDebugger incorporate dynamic state-guided repair, achieving 60% success in fixing vulnerabilities by comparing actual and expected program states [8]. Data-centric self-adaptive agents further enhance adaptability, boosting context learning by 12.2% and training by 19.5% on coding and debugging tasks [9].

Despite these technological advances, Figure 1 reveals that no existing solution combines deep Spring Boot lifecycle awareness with AI-powered real-time suggestions, live stack trace analysis, and comprehensive retrieval-augmented generation from Spring documentation and Stack Overflow. Current tools either focus on generic debugging capabilities or provide limited Spring-specific context without leveraging modern AI techniques for intelligent error analysis and fix generation.

1.2. Research Gap

Current debugging tools in modern IDEs such as IntelliJ IDEA provide only basic stack-trace inspection and breakpoint functionality, requiring developers to manually interpret complex runtime issues, a process that becomes increasingly challenging in large, framework-heavy applications. Despite numerous educational and tool-focused interventions, debugging remains one of the most significant challenges in software development [1].

Recent advances in automated program repair (APR) using large language models demonstrate potential but exhibit critical limitations in practice. Current LLM-based APR systems generate patches that are often too generic, lacking the framework-specific contextualization required for real-world applications such as Spring Boot [2], [4]. These systems tend to overfit common repair patterns rather than adapting to the nuanced requirements of enterprise frameworks [5], [7]. Furthermore, while these models achieve strong performance on benchmark datasets, they experience significant accuracy degradation when applied to complex, framework-heavy codebases [6]. Tools like JetBrains AI provide low-level runtime error analysis but lack the deep accuracy needed for Spring Boot-specific issues and do not incorporate RAG for enhanced suggestions.

In the domain of information retrieval, retrieval-augmented generation (RAG) has emerged as a promising approach to reduce hallucinations by grounding responses in external knowledge sources. However, RAG systems face substantial challenges in software development contexts, specifically in filtering noisy information, integrating heterogeneous sources such as official documentation and community forums, and distinguishing reliable solutions from outdated or incorrect information [3]. While recent advances in quality filtering and adaptive scoring mechanisms show promise [10], these improvements have not been systematically adapted to address the specific requirements of software debugging scenarios.

This analysis reveals a significant research gap: the need for an IntelliJ-integrated debugging assistant that combines runtime error capture, retrieval from curated Spring Boot knowledge sources via RAG, and framework-specific suggestions using an LLM. Such a system would address the limitations of current IDE tools, including JetBrains AI's low-level and generic approach, while leveraging recent advances in AI-assisted program repair, providing developers with contextually relevant and actionable debugging support.

1.3. Research Problem

The complexity of Spring Boot applications, particularly in handling dependency injection, bean configuration, and lifecycle management, leads to runtime errors that are difficult to diagnose and resolve using existing debugging tools. Developers spend significant time manually analyzing stack traces and searching for solutions on platforms like Stack Overflow, reducing productivity and delaying project timelines [5]. The lack of an integrated, AI-powered tool that understands Spring Boot's lifecycle and provides accurate, context-aware fix suggestions directly within the IDE represents a critical challenge. Tools like JetBrains AI offer low-level runtime analysis but fail to deliver framework-specific accuracy or RAG-enhanced fixes. Current LLM-based APR systems generate generic fixes that fail to address framework-specific nuances, while RAG systems, despite their potential, lack tailored implementations for software debugging [2], [3]. This project aims to develop a plugin that leverages an LLM API and RAG to deliver precise, Spring Boot-specific debugging support, improving efficiency and developer experience in handling runtime errors.

2. OBJECTIVES

2.1. Main Objective

To develop an AI-powered IntelliJ plugin that enhances the debugging process for Spring Boot applications by analyzing runtime stack traces and logs, generating intelligent fix suggestions using an LLM API, and enhancing these suggestions with RAG from external knowledge sources, thereby improving debugging efficiency, accuracy, and developer productivity.

2.2. Specific Objectives

- Develop a plugin architecture that integrates seamlessly with IntelliJ IDE and supports real-time analysis of Spring Boot runtime errors.
- Implement a module using an LLM API (e.g., Claude Sonnet 4) to analyze stack traces and logs, identifying the root cause of errors within the Spring Boot lifecycle.
- Design a RAG system to retrieve and integrate relevant information from external sources, such as Spring documentation and Stack Overflow, to enhance fix suggestions.
- Create a user-friendly UI/UX interface within IntelliJ to present fix suggestions and debugging insights interactively.
- Evaluate the plugin's performance through testing with real-world Spring Boot applications, ensuring accuracy and usability.

3. METHODOLOGY

This project aims to develop an IntelliJ plugin for automated error analysis and fix suggestions in Spring Boot applications using Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG). The methodology focuses on data collection, system design, development tasks, testing, and evaluation, ensuring a systematic approach to achieving the project objectives.

3.1. Materials and Resources

3.1.1. Software:

- IntelliJ IDEA (Ultimate, educational license) for plugin development and testing.
- Spring Boot framework (version 3.3.0+) for creating and testing runtime error scenarios.
- Java 17+ for IntelliJ Plugin + Backend development and Spring Boot integration.
- Python 3.13+ with libraries for RAG Pipeline implementation, embedding generation, and retrieval logic.
- PostgreSQL with pgvector extension for the Knowledge Base, enabling efficient storage and querying of vector embeddings.
- Embeddings model: all-MiniLM-L6-v2 for generating dense vector representations of documents and queries.
- Claude Sonnet 4 API for LLM Model inference, providing grounded, context-aware fix suggestions.
- JUnit 5 for unit and integration testing.

3.1.2. Hardware:

- Development laptops with at least 16GB RAM and 4-core CPUs.
- Cloud resources for:
  - Hosting the RAG backend (PostgreSQL + pgvector).
  - Running embedding generation pipelines at scale.

3.1.3. Data:

- Spring Boot official documentation (guides, API references).
- GitHub issues/discussions from repositories such as spring-projects/spring-boot.
- Stack Overflow threads tagged with *spring-boot* (relevant Q&A).

3.2. Data Collection

3.2.1. RAG Knowledge Base:

Curated from Spring Boot official documentation, GitHub issues/discussions, and relevant Stack Overflow threads. These documents will be preprocessed, chunked, and converted into embeddings for storage in PostgreSQL with pgvector.

3.2.2. Test Dataset:

A test dataset for the proposed feature will be obtained by collecting real-world Spring Boot runtime error logs and stack traces from open-source GitHub projects, online repositories, and controlled test applications. Additional datasets will be generated through intentional misconfiguration of Spring Boot components so that reproducible runtime exceptions are produced. These error traces will then be paired with verified solutions extracted from Spring Boot documentation, Stack Overflow discussions, and GitHub issue fixes to create a benchmark dataset for system evaluation.

3.3. Tasks and Subtasks

Task	Subtasks
Plugin Architecture Design	- Define IntelliJ plugin structure and extension points. - Implement API for capturing stack traces and runtime logs. - Enable communication with backend RAG + LLM services.
RAG System Development	- Collect and preprocess Spring Boot documentation, GitHub issues, and Stack Overflow threads. - Generate embeddings using all-MiniLM-L6-v2 and store in Postgres + pgvector Knowledge Base. - Implement retrieval module in Python for fetching top-k relevant documents via vector search. - Add filtering/ranking mechanisms to reduce irrelevant results.
LLM Integration	- Connect to Claude Sonnet 4 via API - Construct dynamic prompts combining stack traces with retrieved documents. - Handle response parsing and format actionable debugging suggestions.
UI/UX Development	- Design IntelliJ plugin UI mockups. - Implement interactive debugging interface
Testing and Validation	- Run evaluation using test dataset as input. - Measure retrieval accuracy and suggestion quality against annotated fixes. - Conduct unit/integration testing of the plugin. - Perform user acceptance testing with developers.

3.4. Anticipated Outcomes

- Provide accurate, context-aware fix suggestions for common Spring Boot runtime errors.
- Reduce debugging time for common errors by 30–50%, helping developers quickly identify likely solutions.
- Improve fix accuracy through context-aware suggestions integrated directly into the IDE.
- Enhance developer productivity by providing an intuitive interface without enforcing step-by-step debugging.
- Enable real-world adoption of automated suggestion tools in software development teams.

3.5. Architecture Diagram

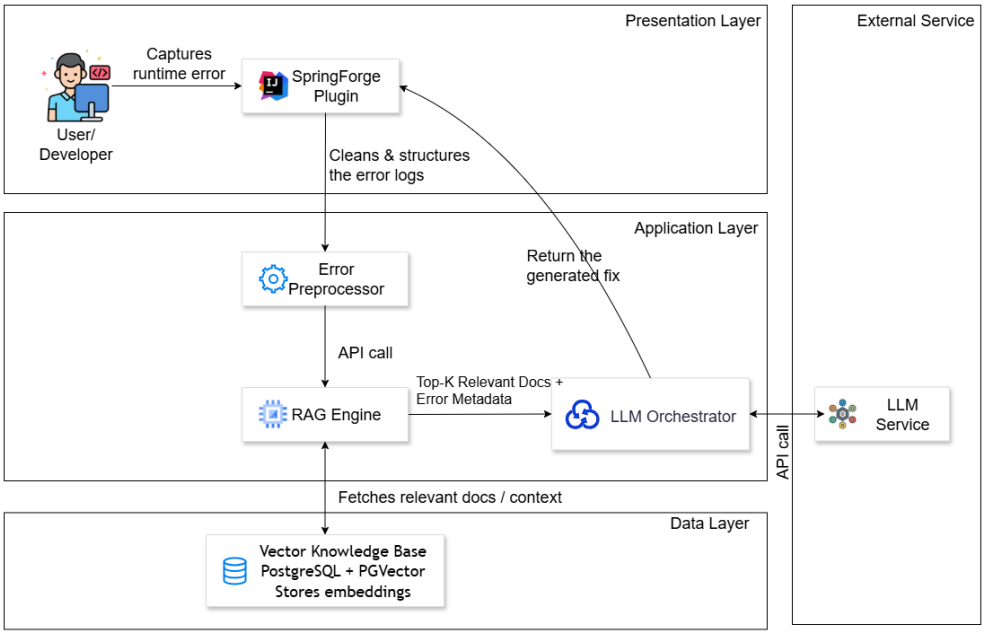

Figure 3.1. Architecture Diagram

4. PROJECT REQUIREMENTS

4.1. Functional Requirements

- The plugin must capture and parse Spring Boot runtime stack traces and logs in real-time from the development environment.
- The system must retrieve relevant information from the curated knowledge base (Spring Boot documentation, GitHub issues, Stack Overflow threads) using embeddings and vector search.
- The Python RAG Pipeline must assemble prompts with the captured error and retrieved documents, send them to the Claude Sonnet 4 API, and obtain debugging suggestions.
- The UI must display suggestions and error details interactively within IntelliJ.

4.2. Non-Functional Requirements

- The plugin must process stack traces within 5 seconds.
- The system must achieve at least 80% accuracy in error classification.
- The UI must be responsive and user-friendly, with a learning curve of less than 10 minutes.

4.3. User Requirements

- Developers should receive clear, actionable fix suggestions.
- The plugin should support Spring Boot versions 2.x and 3.x.

4.4. System Requirements

- Compatible with IntelliJ IDEA Community and Ultimate editions.
- Requires Java 17 or higher for plugin development.
- Python 3.10+ for RAG Pipeline; Postgres + pgvector for Knowledge Base.
- LLM inference via Claude Sonnet 4 API requires at least 16GB RAM for optimal performance.

4.5. Use Cases (Tentative)

- **Use Case 1**: Developer encounters a dependency injection failure; plugin analyzes stack trace, identifies misconfigured bean, and suggests corrected configuration.
- **Use Case 2**: Developer faces a lifecycle error; plugin retrieves relevant Stack Overflow thread and provides a step-by-step fix.

4.6. Test Cases (Tentative)

- **Test Case 1**: Input a stack trace with a known bean misconfiguration; verify the plugin correctly identifies and suggests a fix.

- **Test Case 2**: Test RAG retrieval accuracy with 100 Spring Boot-related Stack Overflow threads.

4.7. Wireframes (Tentative)

- **Wireframe 1:** A sidebar in IntelliJ displaying Spring Boot error details in a structured format (error type, description, possible cause).
- **Wireframe 2:** A pop-up or expandable panel that appears when the user clicks *"How to Fix"*, showing a list of possible fix suggestions, along with contextual references.

4.8. Work Breakdown Structure

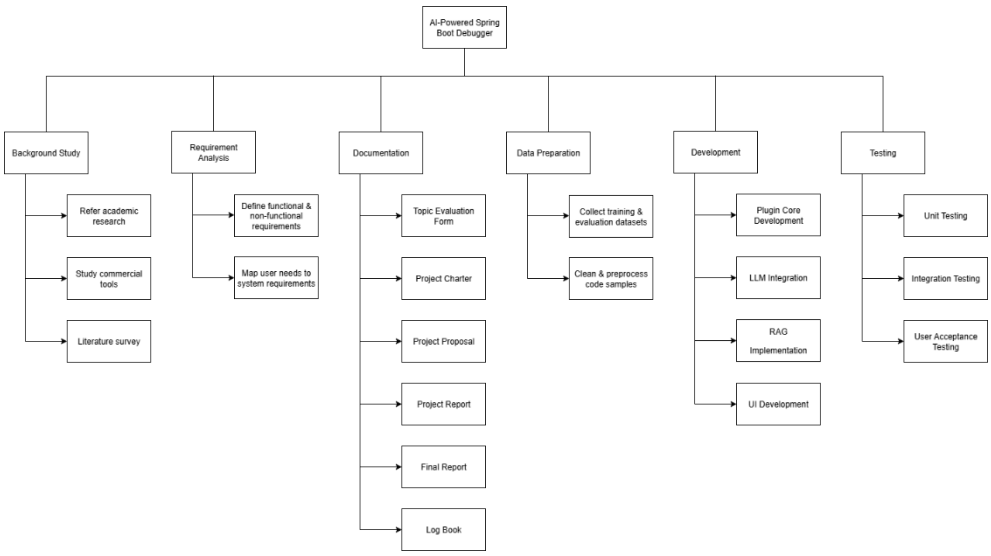

Figure 4.1. Work Breakdown Structure

4.9. Gantt Chart

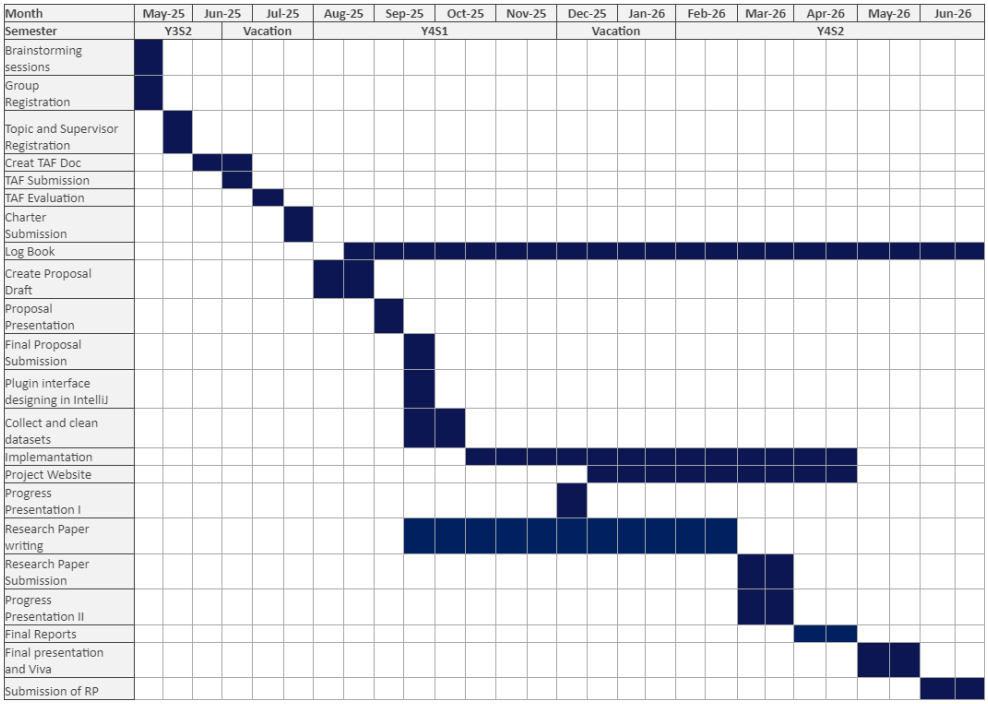

Figure 4.2. Gantt Chart

REFERENCES

[1] S. Yang, M. Baird, E. O'Rourke, K. Brennan, and B. Schneider, "Decoding debugging instruction: A systematic literature review of debugging interventions," *ACM Trans. Comput. Educ.*, vol. 24, no. 4, pp. 1–26, 2024, doi: 10.1145/3688809.

[2] C. S. Xia and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proc. 45th Int. Conf. Softw. Eng. (ICSE)*, Melbourne, VIC, Australia, 2023, pp. 1482–1494, doi: 10.1109/ICSE48619.2023.00127.

[3] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking large language models in retrieval-augmented generation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 16, Vancouver, Canada, 2024, pp. 17754–17762, doi: 10.1609/aaai.v38i16.29723.

[4] B. Yang, Z. Cai, F. Liu, B. Le, L. Zhang, T. F. Bissyandé, Y. Liu, and H. Tian, "A survey of LLM-based automated program repair: Taxonomies, design paradigms, and applications," *arXiv preprint arXiv:2506.23749*, 2025. [Online]. Available: https://arxiv.org/abs/2506.23749

[5] A. Koyuncu, K. Liu, T. F. Bissyandé, D. Kim, J. Klein, M. Monperrus, and Y. Le Traon, "FixMiner: Mining relevant fix patterns for automated program repair," *Empir. Softw. Eng.*, vol. 25, no. 3, pp. 1980–2024, 2020, doi: 10.1007/s10664-019-09780-z.

[6] F. Li, J. Jiang, J. Sun, and H. Zhang, "Evaluating the generalizability of LLMs in automated program repair," *arXiv preprint arXiv:2503.09217*, 2025. [Online]. Available: https://arxiv.org/abs/2503.09217

[7] F. V. Ruiz, M. Hort, and L. Moonen, "The art of repair: Optimizing iterative program repair with instruction-tuned models," *arXiv preprint arXiv:2505.02931*, 2025. [Online]. Available: https://arxiv.org/abs/2505.02931

[8] Z. Liu, Y. Ma, J. Xu, J. Ai, X. Gao, H. Sun, and A. Roychoudhury, "Agent that debugs: Dynamic state-guided vulnerability repair," *arXiv preprint arXiv:2504.07634*, 2025. [Online]. Available: https://arxiv.org/abs/2504.07634

[9] H. Su, R. Sun, J. Yoon, P. Yin, T. Yu, and S. Ö. Arık, "Learn-by-interact: A data-centric framework for self-adaptive agents in realistic environments," *arXiv preprint arXiv:2501.10893*, 2025. [Online]. Available: https://arxiv.org/abs/2501.10893

[10] Rangan, Keshav & Yin, Yiqiao. (2024). A fine-tuning enhanced RAG system with quantized influence measures as AI judge. Scientific Reports. 14. 10.1038/s41598-024-79110-x.